```
"""
generate_grus_keys.py
=====================
GRUS Cryptographic Key Generator — One-Time Setup
Specification: GRUS-SEAL-001-v1.0 Section 1

Generates the LLC's ed25519 signing keypair used by the audit pipeline
to seal certification records. Run ONCE when initializing the canonical
audit engine deployment.

USAGE:
    python3 generate_grus_keys.py [--output-dir /etc/grus]

Outputs:
    private_key.pem  — kept secret by the LLC, used by audit_runner.py
    public_key.pem   — published at grus.utility/public-key for verification
    fingerprint.txt  — SHA-256 fingerprint of the public key for archival reference

WARNING:
    The private key MUST be:
      - generated on a secure, offline machine if possible
      - stored with restrictive file permissions (0600)
      - backed up in encrypted form to multiple secure locations
      - NEVER committed to a git repository
      - NEVER published anywhere

    The public key MUST be:
      - published at the canonical URL grus.utility/public-key
      - mirrored to GitHub, Zenodo, and Internet Archive
      - referenced in every certification record's verification instructions

KEY ROTATION:
    If the private key is ever compromised, follow GRUS-SEAL-001-v1.0
    Section 7 (Public Key Rotation Policy):
      1. Publish 30-day rotation notice on all four mirror locations
      2. Run this script again to generate a NEW keypair
      3. Move the OLD private_key.pem to historical_keys/
      4. Use the new key for all forward audits
      5. Old certifications remain verifiable under the historical key

Published by: Green Recursive Utility Service LLC
"""

import os
import sys
import argparse
import hashlib
from datetime import datetime, timezone
```

```
try:
    from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
    from cryptography.hazmat.primitives import serialization
except ImportError:
    print("ERROR: cryptography library not installed.")
    print("Install: pip install cryptography")
    sys.exit(1)


def generate_keypair(output_dir: str, force: bool = False) -> dict:
    """
    Generate the LLC's ed25519 signing keypair.

    Args:
        output_dir: directory where private_key.pem and public_key.pem are written
        force: if True, overwrite existing keys (DANGEROUS — only for first run)

    Returns:
        dict with paths and fingerprint
    """
    os.makedirs(output_dir, exist_ok=True, mode=0o700)

    private_path = os.path.join(output_dir, "private_key.pem")
    public_path = os.path.join(output_dir, "public_key.pem")
    fingerprint_path = os.path.join(output_dir, "fingerprint.txt")

    if os.path.exists(private_path) and not force:
        print(f"ERROR: {private_path} already exists.")
        print("Refusing to overwrite. If you intend to rotate keys, see")
        print("GRUS-SEAL-001-v1.0 Section 7 (Public Key Rotation Policy).")
        print("If this is a fresh setup and the existing key is unused, pass --force.")
        sys.exit(2)

    # Generate the keypair
    private_key = Ed25519PrivateKey.generate()
    public_key = private_key.public_key()

    # Serialize private key (PEM format, no passphrase by default)
    private_pem = private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption(),
    )
    # Serialize public key (PEM format)
    public_pem = public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo,
    )
```

```
    # Write private key with restrictive permissions
    with open(private_path, "wb") as f:
        f.write(private_pem)
    os.chmod(private_path, 0o600)

    # Write public key (readable by all)
    with open(public_path, "wb") as f:
        f.write(public_pem)
    os.chmod(public_path, 0o644)

    # Compute public-key fingerprint (SHA-256 of the PEM bytes)
    fingerprint = hashlib.sha256(public_pem).hexdigest()
    timestamp = datetime.now(timezone.utc).isoformat()

    # Write fingerprint record
    fingerprint_record = (
        f"GRUS Public Key Fingerprint\n"
        f"============================\n"
        f"Issued by:   Green Recursive Utility Service LLC\n"
        f"State:       Texas Limited Liability Company\n"
        f"Generated:   {timestamp}\n"
        f"Algorithm:   ed25519\n"
        f"Format:      PKCS#8 / SubjectPublicKeyInfo PEM\n"
        f"\n"
        f"SHA-256:     {fingerprint}\n"
        f"\n"
        f"Public Key:\n"
        f"-----------\n"
        f"{public_pem.decode('ascii')}\n"
        f"\n"
        f"Verification: Any third party may verify any GRUS-AUDIT-NNNN\n"
        f"certification record by:\n"
        f"  1. Downloading public_key.pem from grus.utility/public-key\n"
        f"  2. Confirming the SHA-256 matches the value above\n"
        f"  3. Running ed25519 signature verification per GRUS-SEAL-001-v1.0\n"
    )
    with open(fingerprint_path, "w") as f:
        f.write(fingerprint_record)
    os.chmod(fingerprint_path, 0o644)

    return {
        "private_path": private_path,
        "public_path": public_path,
        "fingerprint_path": fingerprint_path,
        "fingerprint_sha256": fingerprint,
        "generated": timestamp,
    }


def main():
```

```
    ap = argparse.ArgumentParser(
        description="Generate the LLC's ed25519 signing keypair for the GRUS audit pipeline."
    )
    ap.add_argument("--output-dir", default="/etc/grus",
                    help="Directory for private_key.pem and public_key.pem (default:
/etc/grus)")
    ap.add_argument("--force", action="store_true",
                    help="Overwrite existing keys (DANGEROUS — see Section 7 rotation policy)")
    args = ap.parse_args()

    print("=" * 70)
    print(" GRUS CRYPTOGRAPHIC KEY GENERATOR — ONE-TIME SETUP")
    print(" Green Recursive Utility Service LLC")
    print(" Specification: GRUS-SEAL-001-v1.0")
    print("=" * 70)
    print()

    result = generate_keypair(args.output_dir, force=args.force)

    print(f"✓ Private key written: {result['private_path']}")
    print(f"  Permissions:         0600 (LLC use only)")
    print()
    print(f"✓ Public key written:  {result['public_path']}")
    print(f"  Permissions:         0644 (world-readable)")
    print()
    print(f"✓ Fingerprint record:  {result['fingerprint_path']}")
    print(f"  SHA-256:             {result['fingerprint_sha256']}")
    print(f"  Generated:           {result['generated']}")
    print()
    print("—" * 70)
    print(" REQUIRED NEXT STEPS")
    print("—" * 70)
    print()
    print(" 1. Publish public_key.pem at the canonical URL:")
    print("    https://grus.utility/public-key")
    print()
    print(" 2. Mirror public_key.pem to:")
    print("    - github.com/grus-utility/public-key")
    print("    - Zenodo community 'grus-utility' (record 0001)")
    print("    - Internet Archive Wayback Machine snapshot")
    print()
    print(" 3. Publish the fingerprint SHA-256 in:")
    print("    - The GRUS Public Charter document set")
    print("    - The first press release")
    print("    - The README of every official LLC repository")
    print()
    print(" 4. Back up private_key.pem in encrypted form to:")
    print("    - At least two secure offline locations")
    print("    - One physical-medium backup (encrypted USB or HSM)")
    print()
```

```
    print(" 5. NEVER commit private_key.pem to a git repository.")
    print(" 6. NEVER publish private_key.pem anywhere.")
    print()
    print("—" * 70)
    print(" Setup complete. The audit pipeline is now ready to issue")
    print(" cryptographically sealed validated-fact certifications.")
    print("—" * 70)


if __name__ == "__main__":
    main()
```